

ZZX OS 操作系统技术手册

作者: TheodoreZheng

版本: v1.0.0

日期: 2026 年 9 月

第一章 系统概述与设计理念

1.1 系统简介

ZZX OS 是一款基于 x86 裸机 (Bare-Metal) 架构完全采用 16 位 x86 汇编语言 (Intel 语法 NASM 编译器) 自主研发的极简微型操作系统内核。系统脱离了对现代庞大操作系统 (如 Linux、Windows) 及其 C 运行时库的依赖, 直接在 x86 计算机实模式 (Real Mode) 下运行。系统紧凑地整合了 MBR Bootloader 引导程序、POST 开机自检引擎、开机动态 Logo 动画、TUI (Text-based User Interface) 双栏交互控制台 Shell, 以及基于 VGA Standard Mode 13h 的 3D 线框图形渲染引擎。

1.2 设计目标与原则

- 极致轻量化:** 系统总体尺寸限制在数 KB 级别, 能够完整放入一张 1.44MB 标准软盘或嵌入式 ROM 中。
- 零依赖与纯汇编:** 无任何外部动态库、静态库或高层次语言运行时支持, 直接通过 x86 汇编指令与 BIOS 中断驱动硬件。
- 确定性与高实时性:** 单任务架构, 消除了进程切换开销与复杂的页表映射, 指令执行路径完全可控。
- 丰富的交互体验:** 在受限的 80x25 文本模式下, 利用直接显存映射 (Direct Video Memory Mapping) 构建现代化的双栏 TUI 界面与状态监控系统。

1.3 系统架构图与模块划分

ZZX OS 由以下四个核心模块构成:

- 引导与初始化模块 (Bootloader & Post Engine):** 负责软盘/磁盘扇区读取、段寄存器重定位、显存初始化与 POST 自检。

2. **TUI 界面与渲染子系统 (TUI Framework & Text Memory Engine):** 控制 80x25 VGA 文本缓冲区 (0xB8000), 实现字符双栏分割、状态栏刷屏与动态焦点切换。
3. **命令解析与 Shell 执行子系统 (Command Interpreter Shell):** 处理键盘中断 INT 0x16、维护输入缓冲区、执行字符串匹配算法并调用对应内核功能。
4. **VGA Mode 13h 3D 图形渲染器 (Mode 13h Graphical Engine):** 切换 320x200 256 色图形模式, 映射 0xA0000 显存, 实现实时三维点线变换与旋转绘制。

第二章 引导流程与系统初始化 (Bootloader & POST)

2.1 MBR 引导程序架构 (boot.asm)

当计算机上电启动时, 主板 BIOS 完成基础硬件检测后, 会将存储设备的第一扇区 (0 柱面、0 磁头、1 扇区, 共 512 字节) 加载至内存物理地址 0x0000:0x7C00 处, 并跳转至该地址开始执行。

boot.asm 的主要执行流程如下:

1. **寄存器与栈初始化:** 设置段寄存器 DS = 0, ES = 0, SS = 0, 并将栈指针 SP 初始化为 0x7C00。
2. **显示引导日志:** 通过调用 BIOS 中断 INT 0x10 (AH=0x0E) 打印 Bootloader 版本信息及内核加载提示。
3. **重置与读取磁盘:** 调用 BIOS INT 0x13 (AH=0x00, DL=0x00) 重置软盘驱动器。随后调用 INT 0x13 (AH=0x02), 读取 18 个扇区 (约 9KB) 至物理地址 0x10000。
4. **容错与重试机制:** 内置 3 次重试计数器。若 3 次均失败, 输出 [FAILED] Disk read error! 并执行 HLT 停机。
5. **远跳转控制转移:** 磁盘读取成功后, 执行 jmp 0x1000:0x0000, 控制权移交至内核入口。

2.2 内核初始化与 POST 自检流程 (kernel.asm)

内核入口 kernel_main 获得控制权后, 立即展开系统初始化:

1. **显存段基址绑定:** 将段寄存器 GS 设置为 0xB800, 该段直连 VGA 彩色文本模式显存。

2. 隐藏硬件光标: 通过 `INT 0x10 (AH=0x01, CX=0x2607)` 修正 VGA 硬件光标, 防止屏幕闪烁。
3. 阶段 1: 动画 Logo 展演: 调用 `draw_boot_logo_animated` 引擎。利用 BIOS `INT 0x15 (AH=0x86)` 延时例程实现色彩脉冲闪烁。
4. 阶段 2: POST 自检 (Power-On Self-Test): 循环遍历 `post_messages` 数组, 输出 15 项硬件与内核状态自检信息 (CPU、内存、中断向量表、RTC 等)。
5. 阶段 3: 进入控制台主界面: 自检完成后, 调用 `draw_split_screen` 构建 TUI 分割屏界面, 启动 Shell 监听。

第三章 TUI 架构与控制台交互引擎

3.1 80x25 文本显存直接映射原理

ZZX OS 采用直接操作 `0xB8000` 显存地址的高效机制。显存物理地址计算公式:

```
Offset = (Row * 80 + Column) * 2
```

- `Offset + 0`: ASCII 字符。
- `Offset + 1`: 前景色与背景色属性 (例如 `0x0F` 表示黑底亮白字)。

3.2 界面布局与双栏分割设计

TUI 主界面由四大功能区域构成:

- 左侧核心控制台 (Column 0-38): Shell 交互区域, 处理命令输入与响应。
- 中央分割线 (Column 40): 写入 `0x08B3` 字符实现视觉隔离。
- 右侧存储模块映射 (Column 42-79): 列表形式展示物理存储块状态。
- 底部系统状态栏 (Row 24): 蓝底白字, 显示热键提示与当前焦点状态。

3.3 交互控制与键盘中断处理

系统通过 BIOS 中断 `INT 0x16` 实现键盘捕获:

- 焦点切换: 按 `Alt+B` 触发 `active_focus` 翻转, 并实时更新底部状态栏。
- Shell 输入逻辑: 限制 16 字节缓冲区, 支持退格 (Backspace) 与回车 (Enter) 执行分发。

- **存储区交互**: 在右侧焦点下，支持方向键高亮移动。

第四章 内核 Shell 指令集

指令名称	功能类别	详细描述
<code>guide</code>	基础导航	输出常用指令导航列表
<code>info</code>	系统信息	显示操作系统基本架构标识
<code>sysver</code>	版本查询	显示内核构建版本号 (KRNL-9042-XETA)
<code>memory</code>	硬件监控	查询 640K 基础内存与 15M 扩展内存状态
<code>storage</code>	硬件监控	查询第一存储块容量与读写属性
<code>hardware</code>	硬件监控	查看 CPU 仿真引擎模式
<code>datetime</code>	时间同步	校准并显示 RTC 实时时钟同步状态
<code>process</code>	进程管理	列出当前运行进程 (KERNEL / SHELL)
<code>diag</code>	系统诊断	触发内核级系统全面自检
<code>wipe</code>	界面控制	清除左侧控制台 Shell 的历史输出
<code>beep</code>	多媒体	驱动主板蜂鸣器发出单音
<code>render</code>	图形引擎	切换至 VGA Mode 13h 执行 3D 图形 Demo
<code>lock</code>	安全与重置	锁屏并重新进入开机自检流程
<code>poweroff</code>	电源管理	停止系统运行并进入低功耗休眠态

第五章 VGA Mode 13h 3D 图形渲染引擎

5.1 Mode 13h 图形模式原理

当用户输入 `render` 命令时，内核调用 `do_3d_demo`:

1. 模式切换: 调用 `INT 0x10` (`AX=0x0013`) 进入 320x200 256 色图形模式。
2. 显存地址映射: 段寄存器 `ES` 指向 `0xA000`。
3. 像素定位: $\text{Offset} = Y * 320 + X$ 。

5.2 3D 线框与矩阵投影算法

- 帧缓冲区清除: 利用 `REP STOSW` 指令快速填零清屏。
- 旋转矩阵变换: 利用 `imul` 与 `idiv` 指令计算旋转:
$$X' = X \cdot \cos(\theta) - Z \cdot \sin(\theta)$$
- 投影公式: $X_{\text{screen}} = 160 + \frac{X' \cdot 50}{100}$ 。
- 绘制算法: `draw_line` 例程采用坐标插值算法向显存连续写入颜色值。

第六章 系统构建、编译与仿真部署

6.1 开发环境要求

- 汇编编译器: NASM v2.15+
- 仿真器: QEMU (qemu-system-i386) 或 VirtualBox

6.2 构建脚本 (`run.bat`)

```
nasm -f bin boot.asm -o boot.bin
```

```
nasm -f bin kernel.asm -o kernel.bin
```

```
# 合并二进制文件生成软盘镜像
```

```
cmd /c "copy /b boot.bin + kernel.bin + desktop.bin os.img"
```

```
# 启动 QEMU 仿真器
```

```
qemu-system-i386 -fda os.img
```

第七章 维护与故障排查

7.1 常见引导错误

- 现象: [FAILED] Disk read error!
- 解决: 检查 boot.asm 中 INT 0x13 读取的扇区数 AL 是否与镜像文件大小匹配。

7.2 显存乱码或光标残留

- 现象: TUI 界面出现字符偏移或硬件光标残留。
- 解决: 确保已调用 INT 0x10 (AH=0x01, CX=0x2607)。检查显存寻址公式是否严格遵循 $\text{Row} * 160 + \text{Col} * 2$ 。

文档确认人: TheodoreZheng
最后审核日期: 2026 年 9 月 1 日